UNIVERSIDAD DE CARABOBO
FACULTAD DE CIENCIAS Y TECNOLOGÍA
ARQUITECTURA DEL COMPUTADOR

INFORME DE PRÁCTICA DE LABORATORIO 4

Autores:
José Álvarez — Cédula: 31.389.258
José Núñez — Cédula: 31.581.050

---

1. Explique con un diagrama cómo funciona el ciclo de una interrupción de hardware (desde que ocurre el evento hasta que se atiende).

   El ciclo de una interrupción de hardware se compone de las siguientes fases secuenciales:

   a) **Generación del evento:** Un dispositivo periférico (como un teclado o un reloj) genera una señal eléctrica de solicitud de interrupción (IRQ) hacia el controlador de interrupciones.

   b) **Detección y muestreo:** Al finalizar la ejecución de la instrucción actual en el ciclo del procesador, la CPU comprueba las líneas de interrupción. Si las interrupciones están habilitadas, la CPU acepta la solicitud.

   c) **Cambio de contexto y guardado:** El procesador suspende temporalmente el programa principal, guarda el contador de programa actual (en el registro EPC en arquitecturas MIPS) y deshabilita temporalmente nuevas interrupciones para evitar conflictos.

   d) **Salto a la rutina (ISR):** La CPU salta automáticamente a la dirección de memoria asignada a la Rutina de Servicio de Interrupción (ISR).

   e) **Atención y limpieza:** El software de la ISR atiende al periférico (lee datos, limpia banderas de estado) y se restaura el contexto previo para retornar al flujo principal mediante una instrucción de retorno de interrupción.

2. ¿Qué diferencias hay entre gestionar E/S con sondeo y hacerlo con interrupciones?

   La gestión por *sondeo* (polling) requiere que el microprocesador consulte de manera continua y repetitiva el estado del periférico para saber si tiene datos listos, lo que desperdicia ciclos de procesamiento en espera activa. En contraste, la gestión por *interrupciones* libera a la CPU de esta tarea de vigilancia; el procesador ejecuta sus tareas normales de manera ininterrumpida y es el periférico el que avisa a la CPU mediante una señal cuando requiere atención inmediata.

3. ¿Qué ventajas tiene el uso de interrupciones en términos de uso del procesador?

   La principal ventaja es la eficiencia y el aprovechamiento óptimo del tiempo de procesamiento. Al no desperdiciar ciclos en bucles de espera pasiva (sondeo), la CPU puede dedicar el 100 % de su capacidad de cómputo a la ejecución de tareas útiles, respondiendo a los eventos externos únicamente en el instante exacto en que ocurren.

4. ¿Qué registros especiales se utilizan en MIPS32 para gestionar interrupciones?

   En la arquitectura MIPS32, los registros del Coprocesador 0 (CP0) principales para la gestión de excepciones e interrupciones son:

   - **Status (12):** Controla la habilitación global de interrupciones, el modo de operación y las máscaras de interrupción por hardware.
   - **Cause (13):** Almacena información sobre la última excepción o interrupción ocurrida, indicando el código de la causa que la generó.
   - **EPC (14):** *Exception Program Counter*, almacena la dirección de la instrucción en la que ocurrió la interrupción o instrucción que provocó la excepción para poder reanudar el flujo posteriormente.

5. ¿Por qué es necesario guardar el contexto (registros) al entrar en una rutina de servicio?

   Es necesario porque la Rutina de Servicio de Interrupción (ISR) comparte los mismos registros generales del procesador que el programa principal. Si la ISR modifica el contenido de estos registros sin respaldarlos previamente en la pila, al retornar al flujo principal los datos originales se habrían perdido, corrompiendo el estado del programa y provocando fallos de ejecución.

6. Momentos en que pueden generarse excepciones en un sistema MIPS32.

   a) Enumera al menos 4 situaciones en las que se pueda generar una excepción (por ejemplo: desbordamiento aritmético, fallo de dirección, etc.).

      1. Desbordamiento aritmético (Overflow en operaciones con signo).
      2. Instrucción ilegal o no válida.
      3. Fallo de dirección de memoria por alineación incorrecta (Address Error).
      4. Llamada al sistema (Syscall) o punto de interrupción (Breakpoint).

   b) Explica qué etapas del pipeline pueden provocar una excepción y por qué.

      Las excepciones pueden originarse en distintas etapas: la etapa de *Búsqueda (Fetch)* e *Instrucción (Decode)* pueden generar excepciones por direcciones de memoria inválidas o instrucciones no reconocidas. La etapa de *Ejecución (Execute)* es la principal fuente de excepciones internas, como desbordamientos aritméticos o divisiones entre cero.

7. Estrategias de tratamiento de excepciones e interrupciones.

a) Explica las diferencias entre interrupciones y excepciones (¿son síncronas o asíncronas?).

   Las interrupciones son eventos *asíncronos* generados por hardware externo (dispositivos de E/S o temporizadores) independientes de la instrucción que se esté ejecutando en ese momento. Las excepciones son eventos *síncronos* causados directamente por la instrucción que está ejecutándose en el procesador (como un error aritmético o una llamada al sistema).

b) Describe brevemente dos estrategias para tratar excepciones en un sistema MIPS32. ¿Cómo se redirige la ejecución hacia la rutina de servicio? ¿Cuál es la función del registro EPC (Exception Program Counter)?

   Dos estrategias comunes son el manejo centralizado (vectores fijos donde todas las excepciones apuntan a una única dirección de atención que evalúa la causa) y el manejo vectoreado (donde cada excepción posee su propia dirección de salto específica). La ejecución se redirige modificando el flujo del programa hacia una dirección de memoria predeterminada (por ejemplo, en MIPS suele ser `0x80000180`). La función del registro **EPC** es almacenar la dirección de la instrucción interrumpida o afectada para permitir que, una vez solucionada la excepción, el procesador pueda retornar exactamente al punto donde se encontraba.

8. Habilitación de interrupciones en dispositivos y procesador.

   a) Explica cómo se habilitan las interrupciones en: el teclado, la pantalla y el procesador (detalla qué bits del registro Status deben modificarse).

      En los dispositivos periféricos (como teclado y pantalla), las interrupciones se habilitan activando el bit de control de interrupción en su respectivo registro de control (por ejemplo, activando el bit 1 en el registro de control de recepción de teclado `0xffff0000`). En el procesador, se debe configurar el registro *Status* del CP0, habilitando específicamente el bit de habilitación global de interrupciones (IE - Interrupt Enable, bit 0) y los bits correspondientes en la máscara de interrupciones (IM - Interrupt Mask, bits 8-15) para permitir las líneas de hardware correspondientes.

   b) ¿Qué pasaría si habilitamos interrupciones en los dispositivos, pero no en el procesador?

      Si los dispositivos generan señales de interrupción pero el procesador las mantiene deshabilitadas a nivel global (bit IE en 0), la CPU ignorará por completo dichas solicitudes y el sistema operará como si estuviera trabajando mediante sondeo o perdiendo las señales de los periféricos.

9. Procesamiento de interrupciones.

   a) Describe paso a paso qué ocurre cuando se produce una interrupción de reloj: Desde que el evento ocurre hasta que la rutina de servicio termina. ¿Qué registros se guardan y restauran? ¿Qué hace el hardware y qué hace el software (sistema operativo o rutina)?

El reloj genera la señal de interrupción; el hardware la detecta, salva la dirección de retorno en el EPC, cambia el modo de ejecución y salta a la dirección de la ISR. El software (la rutina) guarda el contexto de los registros generales utilizados en la pila, procesa la tarea del temporizador, limpia la bandera de interrupción, restaura los registros de la pila y ejecuta la instrucción de retorno para volver al programa principal.

b) ¿Por qué es importante guardar el contexto (registros generales, EPC, Status) al entrar en la rutina?

Es vital porque garantiza la integridad de la ejecución en curso. Si ocurre otra interrupción anidada o si la rutina modifica valores de registros críticos y del estado del procesador, sin este respaldo se corrompería la información del hilo principal de ejecución.

10. Interrupciones de reloj y control de ejecución.

a) Explica cómo una interrupción de reloj puede usarse para: Evitar que un programa quede en un bucle infinito y finalizar programas que superan un tiempo máximo de ejecución.

El temporizador del sistema genera interrupciones periódicas a intervalos regulares. Un sistema operativo puede programar un contador o cuota de tiempo (*time slice*) para cada proceso; si el programa supera dicho límite sin ceder el control, la interrupción de reloj obliga a suspenderlo, evitando bloqueos por bucles infinitos o terminando el proceso por exceder su tiempo máximo de ejecución.

b) ¿Qué debe hacer el software si el programa finaliza antes de que ocurra la interrupción de reloj?

El software debe cancelar, desarmar o reiniciar el temporizador programado (limpiando los registros de control del timer) para evitar que se genere una interrupción innecesaria o falsa cuando ya no hay un proceso activo al cual atender.

11. Análisis y Discusión de los Resultados.

Durante el desarrollo y pruebas de la práctica en el entorno MARS, se comprobó que la implementación mediante E/S mapeada en memoria (MMIO) permite una correcta interacción síncrona con los periféricos simulados. El primer programa demostró ser eficaz al filtrar y almacenar selectivamente caracteres del rango alfabético mayúsculo dentro de una ventana de tiempo predefinida, descartando adecuadamente el resto de entradas basándose en restricciones estrictas de códigos ASCII. Por otro lado, la simulación del semáforo demostró que el control temporal secuencial y la lectura no bloqueante de eventos de teclado permiten estructurar máquinas de estado robustas y confiables en lenguaje ensamblador MIPS32.